

Dan Jurafsky and James Martin
Speech and Language Processing

Chapter 6:
Vector Semantics, Part II

Distributional similarity based representations

You can get a lot of value by representing a word by means of its neighbors

“You shall know a word by the company it keeps”

(J. R. Firth 1957: 11)



One of the most successful ideas of modern statistical NLP

*...government debt problems turning into **banking** crises as happened in 2009...*

*...saying that Europe needs unified **banking** regulation to replace the hodgepodge...*

*...India has just given its **banking** system a shot in the arm...*

Tf-idf and PPMI are
sparse representations

tf-idf and PPMI vectors are

- **long** (length $|V| = 20,000$ to $50,000$)
- **sparse** (most elements are zero)

Alternative: dense vectors

vectors which are

- **short** (length 50-1000)
- **dense** (most elements are non-zero)

Sparse versus dense vectors

Why dense vectors?

- Short vectors may be easier to use as **features** in machine learning (less weights to tune)
- Dense vectors may **generalize** better than storing explicit counts
- They may do better at capturing synonymy:
 - *car* and *automobile* are synonyms; but are distinct dimensions
 - a word with *car* as a neighbor and a word with *automobile* as a neighbor should be similar, but aren't
- **In practice, they work better**

Word2vec

Popular embedding method

Very fast to train

Code available on the web

Idea: **predict** rather than **count**

Word2vec

- Instead of **counting** how often each word w occurs near "*apricot*"
- Train a classifier on a binary **prediction** task:
 - Is w likely to show up near "*apricot*"?
- We don't actually care about this task
 - But we'll take the learned classifier weights as the word embeddings

Brilliant insight: Use running text as implicitly supervised training data!

- A word s near *apricot*
 - Acts as gold ‘correct answer’ to the question
 - “Is word w likely to show up near *apricot*?”
- No need for hand-labeled supervision
- The idea comes from **neural language modeling**
 - Bengio et al. (2003)
 - Collobert et al. (2011)

Word2vec is a family of algorithms

[Mikolov et al. 2013]

Predict between every word and its context words!

Use CBOW when:

You have a large amount of text and want training to be relatively fast.
You have limited computational resources.

Two algorithms

1. **Skip-grams (SG)**

Predict context words given target (position independent)

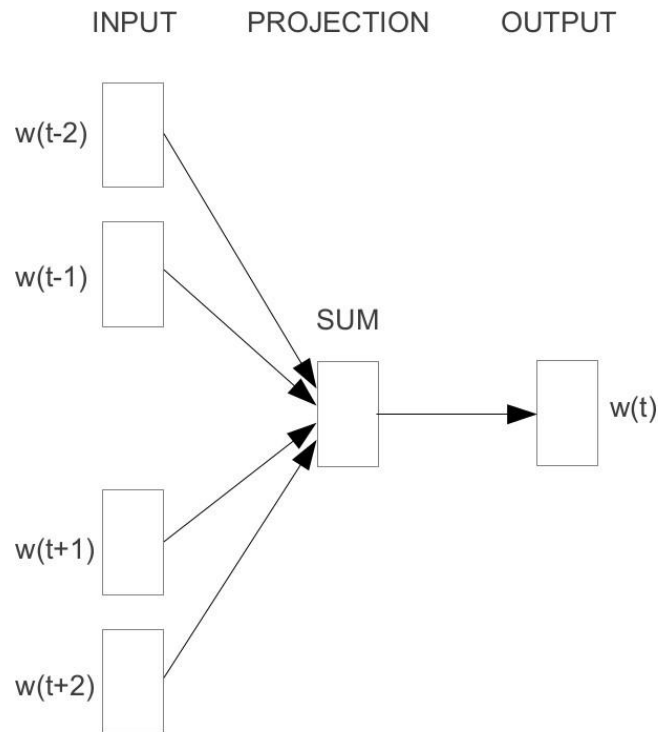
2. **Continuous Bag of Words (CBOW)**

Predict target word from bag-of-words context

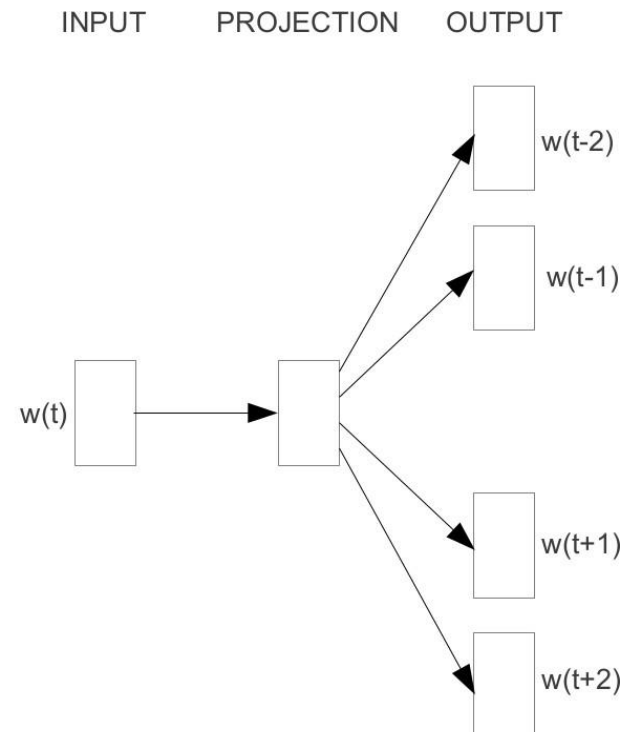
Training methods

1. Hierarchical softmax
2. **Negative sampling**
3. Naïve softmax

Word2vec CBOW (Continuous Bag Of Words) and Skip-gram network architectures



CBOW



Skip-gram

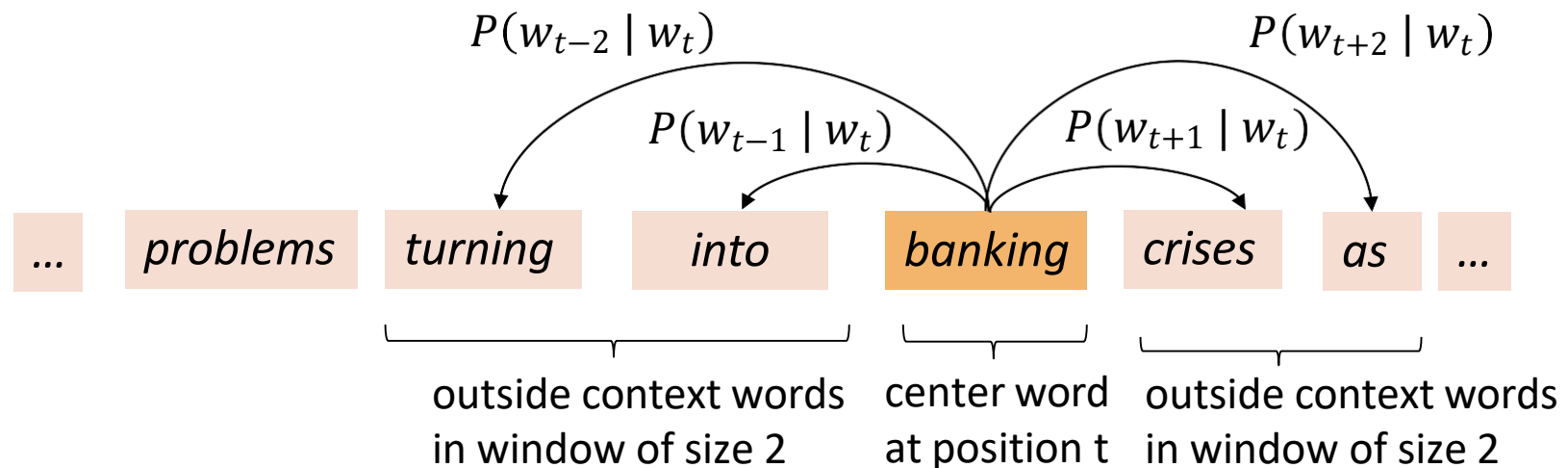
Word2Vec: Skip-Gram Task

Word2vec provides a variety of options. Let's do

- "skip-gram with negative sampling" (SGNS)

Word2Vec Skip-gram Overview

Example windows and process for computing $P(w_{t+j} | w_t)$



WordToVec algorithm

1. Treat the target word and a neighboring context word as positive examples.
2. Use other words not in context as negative samples
3. Use logistic regression to train a classifier to distinguish those two cases
4. Use the learned weights (parameters) of classifier as the embeddings

Setup

Let's represent words as vectors of some length (say 300), randomly initialized.

So we start with $300 * V$ random parameters

Over the entire training set, we'd like to adjust those word vectors such that we

- Maximize the similarity of the **target word, context word** pairs (t,c) drawn from the positive data
- Minimize the similarity of the (t,c) pairs drawn from the negative data.

Learning the classifier

Iterative process.

We'll start with 0 or random weights

Then adjust the word weights to

- make the positive pairs more likely
- and the negative pairs less likely

over the entire training set:

Center word
Context word(s)

x_k $y_{c=1}$ $y_{c=2}$

#1 The quick brown fox jumps over the lazy dog

#2 The quick brown fox jumps over the lazy dog

#3 The quick brown fox jumps over the lazy dog

#4 The quick brown fox jumps over the lazy dog

$y_{c=1}$ $y_{c=2}$ x_k $y_{c=3}$ $y_{c=4}$

#5 The quick brown fox jumps over the lazy dog

#6 The quick brown fox jumps over the lazy dog

#7 The quick brown fox jumps over the lazy dog

#8 The quick brown fox jumps over the lazy dog

$y_{c=1}$ $y_{c=2}$ x_k

#9 The quick brown fox jumps over the lazy dog

one-hot encoding

brown
dog
fox
jumps
lazy
over
quick
the



#1

0	0	1
0	0	0
0	0	0
0	0	0
0	0	0
0	0	0
0	1	0
1	0	0

x_k

$\swarrow$
 $c=1$

$\swarrow$
 $c=2$

#5

0	1	0	0	0
0	0	0	0	0
0	0	1	0	0
1	0	0	0	0
0	0	0	0	0
0	0	0	1	0
0	0	0	0	0
0	0	0	0	1

x_k

$\swarrow$
 $c=1$

$\swarrow$
 $c=2$

$\swarrow$
 $c=3$

$\swarrow$
 $c=4$

#9

0	0	0
1	0	0
0	0	0
0	0	0
0	0	0
0	0	1
0	0	0
0	0	0
0	1	0

x_k

$\swarrow$
 $c=1$

$\swarrow$
 $c=2$

nathanrooy.github.io

#1	natural	language	processing	and	machine	learning	is	fun	and	exciting	#1
	X _k	Y(c=1)	Y(c=2)								
#2	natural	language	processing	and	machine	learning	is	fun	and	exciting	#2
	Y(c=1)	X _k	Y(c=2)	Y(c=3)							
#3	natural	language	processing	and	machine	learning	is	fun	and	exciting	#3
	Y(c=1)	Y(c=2)	X _k	Y(c=3)	Y(c=4)						
#4	natural	language	processing	and	machine	learning	is	fun	and	exciting	#4
		Y(c=1)	Y(c=2)	X _k	Y(c=3)	Y(c=4)					
#5	natural	language	processing	and	machine	learning	is	fun	and	exciting	#5
			Y(c=1)	Y(c=2)	X _k	Y(c=3)	Y(c=4)				
#6	natural	language	processing	and	machine	learning	is	fun	and	exciting	#6
			Y(c=1)	Y(c=2)	X _k	Y(c=3)	Y(c=4)				
#7	natural	language	processing	and	machine	learning	is	fun	and	exciting	#7
				Y(c=1)	Y(c=2)	X _k	Y(c=3)	Y(c=4)			
#8	natural	language	processing	and	machine	learning	is	fun	and	exciting	#8
					Y(c=1)	Y(c=2)	X _k	Y(c=3)	Y(c=4)		
#9	natural	language	processing	and	machine	learning	is	fun	and	exciting	#9
						Y(c=1)	Y(c=2)	X _k	Y(c=3)		
#10	natural	language	processing	and	machine	learning	is	fun	and	exciting	#10
							Y(c=1)	Y(c=2)	X _k		

#	Token	#1				#2				#3					#4					#5				
0	natural	1	0	0	0	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	language	0	1	0	1	0	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0
2	processing	0	0	1	0	0	1	0	1	0	0	0	0	0	0	0	1	0	0	0	1	0	0	0
3	and	0	0	0	0	0	0	1	0	0	0	1	0	1	0	0	0	0	0	0	1	0	0	0
4	machine	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0	1	0	0	0	0	0	0
5	learning	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	1	0	0
6	is	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
7	fun	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
8	exciting	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
		X _k	Y(c=1)	Y(c=2)	X _k	Y(c=1)	Y(c=2)	Y(c=3)	X _k	Y(c=1)	Y(c=2)	Y(c=3)	Y(c=4)	X _k	Y(c=1)	Y(c=2)	Y(c=3)	Y(c=4)	X _k	Y(c=1)	Y(c=2)	Y(c=3)	Y(c=4)	

#	Token	#6					#7					#8					#9					#10		
0	natural	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
1	language	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
2	processing	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
3	and	0	1	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	1	0	
4	machine	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
5	learning	1	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	
6	is	0	0	0	1	0	1	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	
7	fun	0	0	0	0	1	0	0	0	1	0	1	0	0	0	0	0	1	0	0	1	0	0	
8	exciting	0	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	1	1	0	0	0	0	
		X _k	Y(c=1)	Y(c=2)	(c=3	Y(c=4)	X _k	Y(c=1)	(c=2	Y(c=3)	Y(c=4)	X _k	Y(c=1)	(c=2	Y(c=3)	Y(c=4)	X _k	Y(c=1)	(c=2	Y(c=3)	X _k	Y(c=1)	Y(c=2)	

derekchia.com

first and last element in the first training window

1 [Target (*natural*)], [Context (language, processing)]

[list([1, 0, 0, 0, 0, 0, 0, 0, 0])

list([[0, 1, 0, 0, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0, 0, 0]])]

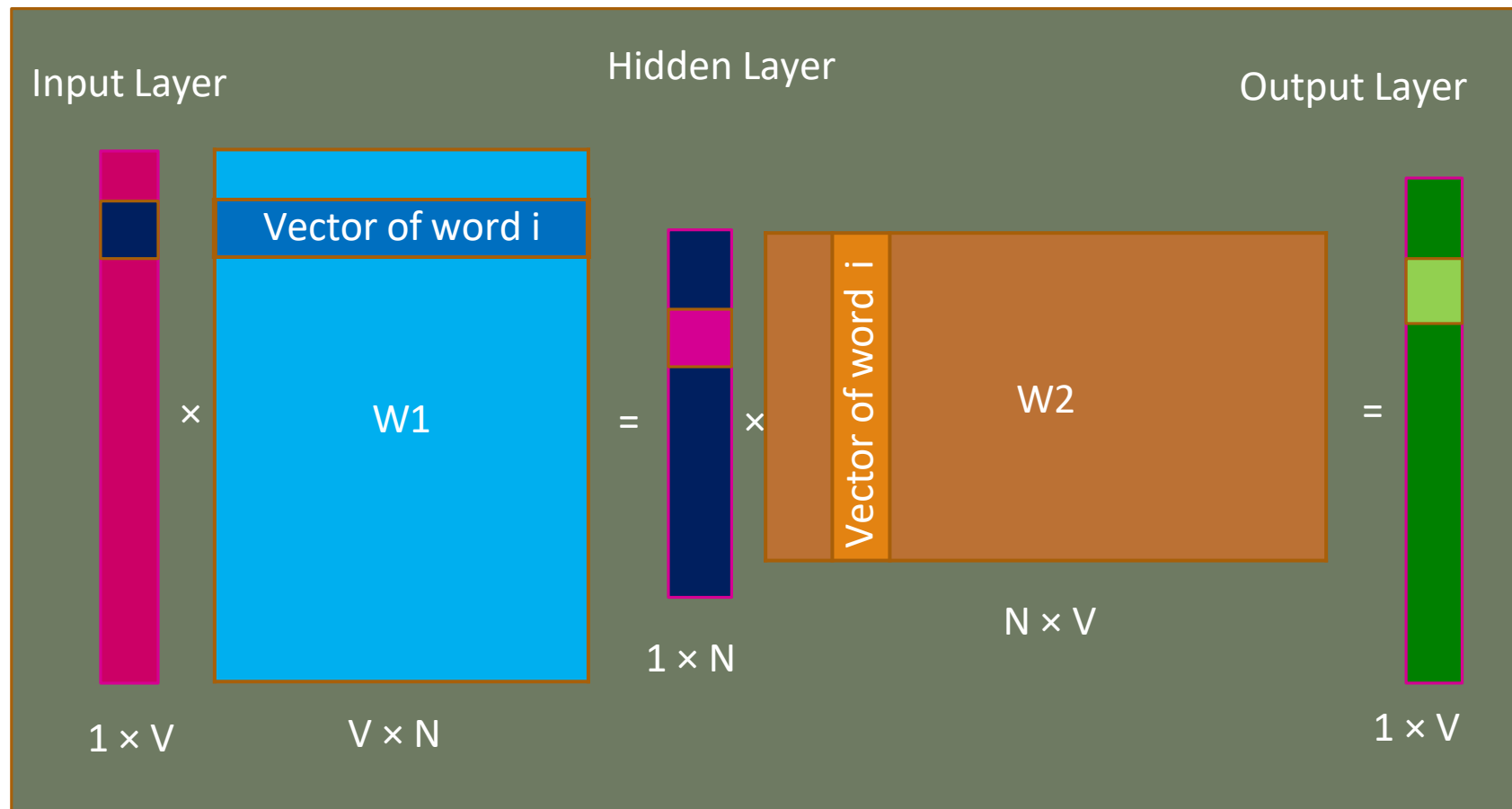
first and last element in the last training window

#10 [Target (*exciting*)], [Context (fun, and)]

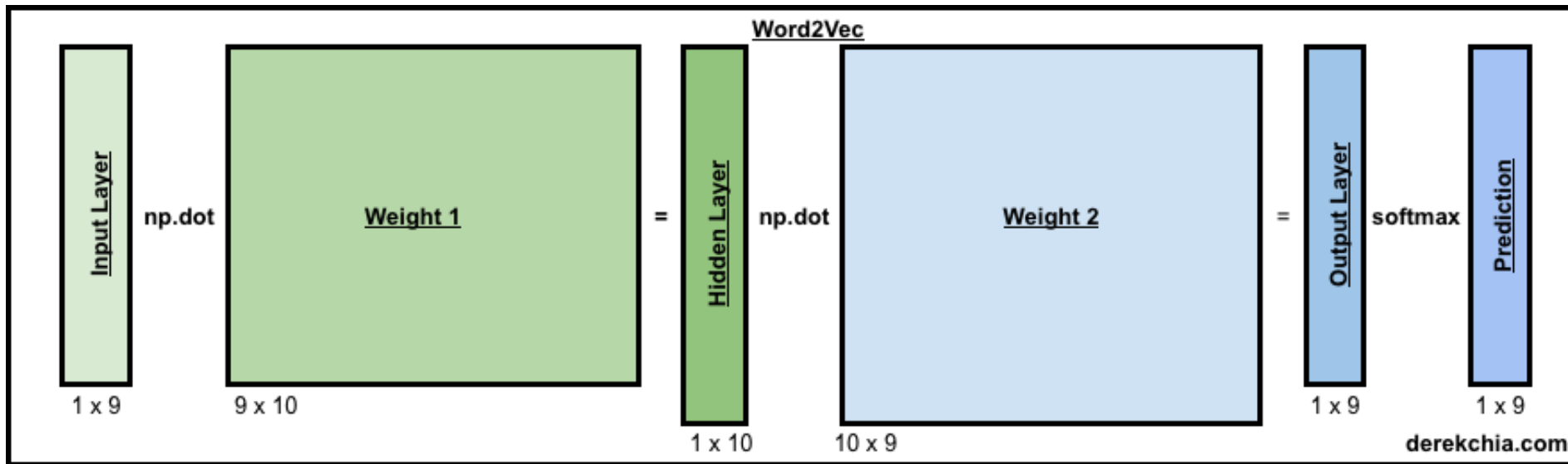
[list([0, 0, 0, 0, 0, 0, 0, 0, 1])

list([[0, 0, 0, 0, 0, 0, 0, 1, 0], [0, 0, 0, 1, 0, 0, 0, 0, 0]])]

N = Dimensions, V = Vocabulary, unique words



Word2Vec — skip-gram network architecture



4. Skip-gram Model Architecture

derekchia.com

Forward Pass for #1

Parameters - Embedding size

10

Random initialisation (Range)

-1 to 1

Calculate Hidden Layer

#	Token	Input - w _t		Weight 1 - W1		Hidden Layer - h
0	natural	1		0.236 -0.962 0.686 0.785 -0.454 -0.833 -0.744 0.677 -0.427 -0.066		0.236 -0.962 0.686 0.785 -0.454 -0.833 -0.744 0.677 -0.427 -0.066
1	language	0		-0.907 0.894 0.225 0.673 -0.579 -0.428 0.685 0.973 -0.070 -0.811		
2	processing	0		-0.576 0.658 -0.582 -0.112 0.662 0.051 -0.401 -0.921 -0.158 0.529		
3	and	0		0.517 0.436 0.092 -0.835 -0.444 -0.905 0.879 0.303 0.332 -0.275		
4	machine	0	np.dot	0.859 -0.890 0.651 0.185 -0.511 -0.456 0.377 -0.274 0.182 -0.237	=	
5	learning	0		0.368 -0.867 -0.301 -0.222 0.630 0.808 0.088 -0.902 -0.450 -0.408		
6	is	0		0.728 0.277 0.439 0.138 -0.943 -0.409 0.687 -0.215 -0.807 0.612		
7	fun	0		0.593 -0.699 0.020 0.142 -0.638 -0.633 0.344 0.868 0.913 0.429		
8	exciting	0		0.447 -0.810 -0.061 -0.495 0.794 -0.064 -0.817 -0.408 -0.286 0.149		
		1 x 9		9 x 10		1 x 10

Calculate y_{pred}

Hidden Layer - h	Weight 2 - W2		Output Layer	Softmax - y_pred
0.236 -0.962 0.686 0.785 -0.454 -0.833 -0.744 0.677 -0.427 -0.066 np.dot 1 x 10	-0.868 -0.406 -0.288 -0.016 -0.560 0.179 0.099 0.438 -0.551 -0.395 0.890 0.685 -0.329 0.218 -0.852 -0.919 0.665 0.968 -0.128 0.685 -0.828 0.709 -0.420 0.057 -0.212 0.728 -0.690 0.881 0.238 0.018 0.622 0.936 -0.442 0.936 0.586 -0.020 -0.478 0.240 0.820 -0.731 0.260 -0.989 -0.626 0.796 -0.599 0.679 0.721 -0.111 0.083 -0.738 0.227 0.560 0.929 0.017 -0.690 0.907 0.464 -0.022 -0.005 -0.004 -0.425 0.299 0.757 -0.054 0.397 -0.017 -0.563 -0.551 0.465 -0.596 -0.413 -0.395 -0.838 0.053 -0.160 -0.164 -0.671 0.140 -0.149 0.708 0.425 0.096 -0.995 -0.313 0.881 -0.402 -0.631 -0.660 0.184 0.487 10 x 9	=	1.258 -1.369 -1.828 1.196 0.545 1.113 1.333 -1.528 -2.335 1 x 9	0.218 0.016 0.010 0.205 0.107 0.189 0.235 0.013 0.006 1 x 9

WordToVec Example

WordToVec Example

$N=3, V=5$

(w_1)

$[0 \ 0 \ 1 \ 0 \ 0]$ 1×5

$\begin{bmatrix} w_{11}^{(1)} & w_{12}^{(1)} & w_{13}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} & w_{23}^{(1)} \\ w_{31}^{(1)} & w_{32}^{(1)} & w_{33}^{(1)} \\ w_{41}^{(1)} & w_{42}^{(1)} & w_{43}^{(1)} \\ w_{51}^{(1)} & w_{52}^{(1)} & w_{53}^{(1)} \end{bmatrix}$ 5×3

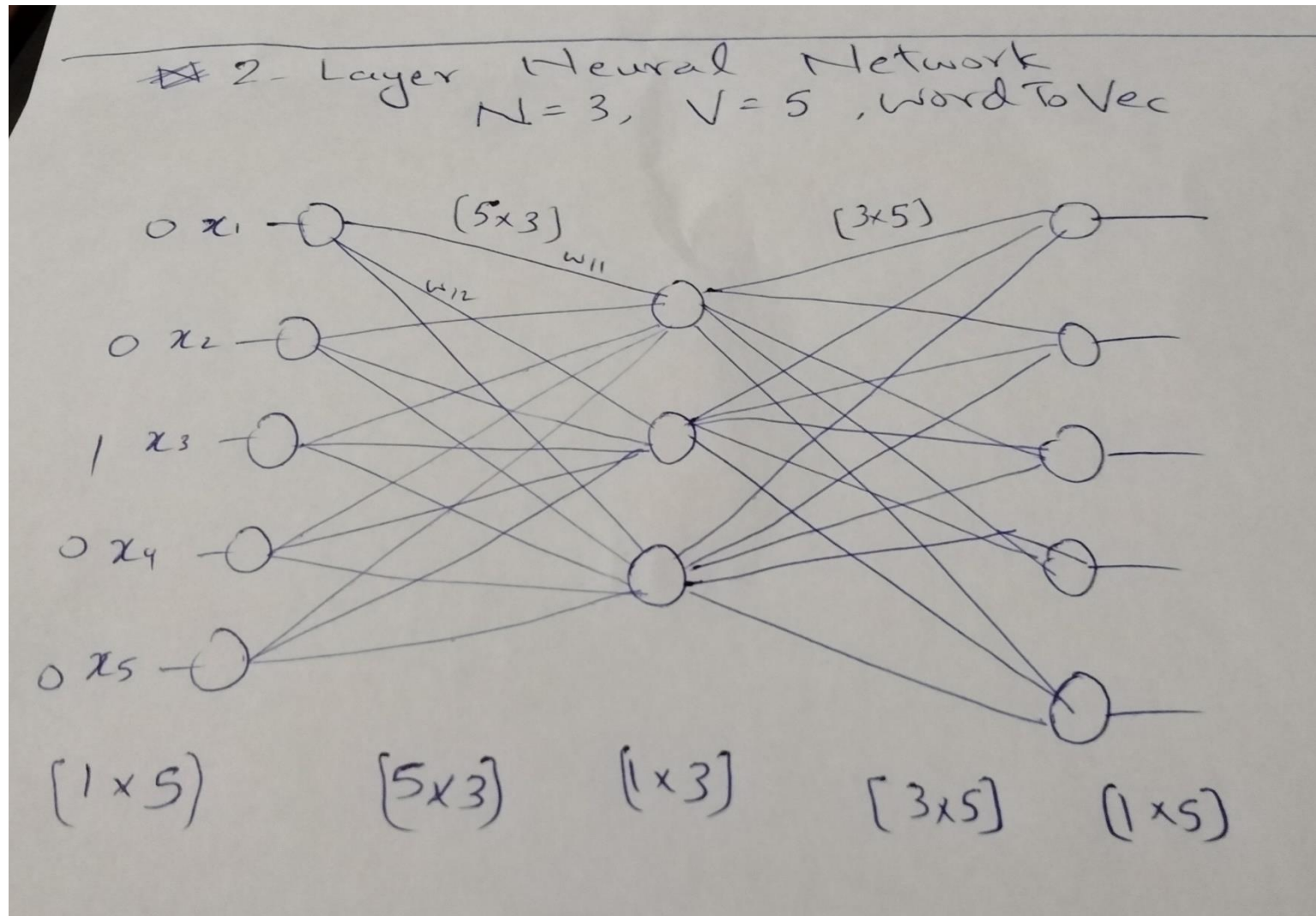
$= [w_{31} \ w_{32} \ w_{33}]$ 1×3

(w_2)

$\begin{bmatrix} w_{11}^{(2)} & w_{12}^{(2)} & w_{13}^{(2)} & w_{14}^{(2)} & w_{15}^{(2)} \\ w_{21} & w_{22} & w_{23} & w_{24} & w_{25} \\ w_{31} & w_{32} & w_{33} & w_{34} & w_{35} \end{bmatrix}$ 3×5

$= \begin{bmatrix} \vdots \\ \vdots \\ \vdots \end{bmatrix}$ 1×5

WordToVec Example 2 layer neural network



Word To Vec Example

$N=3, V=5$

(w_1)

$$\begin{bmatrix} 0 & 0 & 1 & 0 & 0 \end{bmatrix}_{1 \times 5} \cdot \begin{bmatrix} w_{11}^{(1)} & w_{12}^{(1)} & w_{13}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} & w_{23}^{(1)} \\ w_{31}^{(1)} & w_{32}^{(1)} & w_{33}^{(1)} \\ w_{41}^{(1)} & w_{42}^{(1)} & w_{43}^{(1)} \\ w_{51}^{(1)} & w_{52}^{(1)} & w_{53}^{(1)} \end{bmatrix}_{5 \times 3} = \begin{bmatrix} w_{31} & w_{32} & w_{33} \end{bmatrix}_{1 \times 3}$$

(w_2)

$$\begin{bmatrix} w_{11}^{(2)} & w_{12}^{(2)} & w_{13}^{(2)} & w_{14}^{(2)} & w_{15}^{(2)} \\ w_{21}^{(2)} & w_{22}^{(2)} & w_{23}^{(2)} & w_{24}^{(2)} & w_{25}^{(2)} \\ w_{31}^{(2)} & w_{32}^{(2)} & w_{33}^{(2)} & w_{34}^{(2)} & w_{35}^{(2)} \end{bmatrix}_{3 \times 5}$$

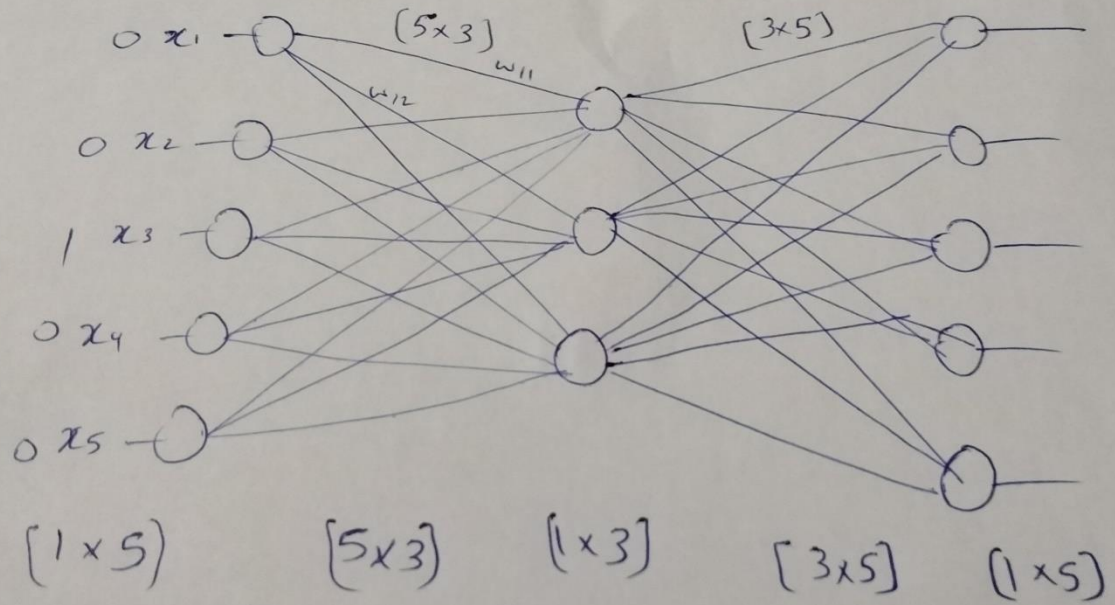
1×3

3×5

$$= \begin{bmatrix} \vdots \\ \vdots \\ \vdots \end{bmatrix}_{1 \times 5}$$

2-Layer Neural Network

$N=3, V=5, \text{wordToVec}$



Skip-Gram Training Data

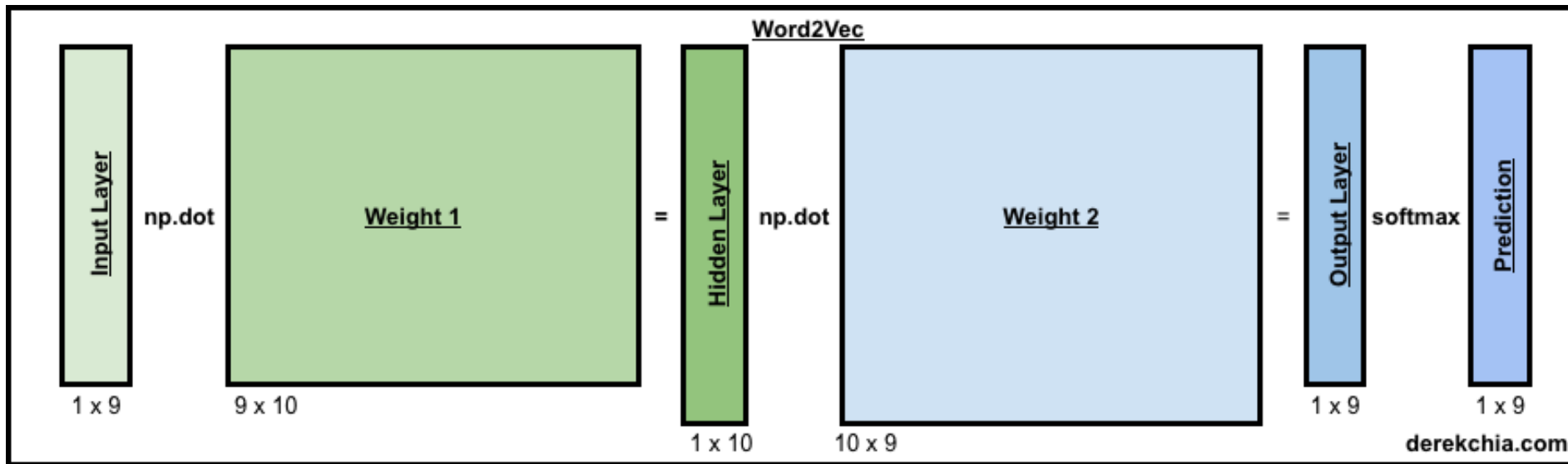
Training sentence:

... lemon, a tablespoon of apricot jam a pinch ...

c1 c2 target c3 c4

Assume context words are those in +/- 2
word window

Motivation for Negative Sampling (weight 2 matrix is updated for every example)



WordToVec Goal

Given a tuple (t, c) = target, context

- (*apricot*, *jam*)
- (*apricot*, *aardvark*)

Return probability that c is a real context word:

$$P(+ | t, c)$$

$$P(- | t, c) = 1 - P(+ | t, c)$$

Skip-gram algorithm

1. Treat the target word and a neighboring context word as positive examples.
2. Randomly sample other words in the lexicon to get negative samples
3. Use logistic regression to train a classifier to distinguish those two cases
4. Use the weights as the embeddings

Skip-Gram Training Data with Negative Sampling

Training sentence:

... lemon, a tablespoon of **apricot** jam a pinch ...

c1 c2 t c3 c4

Training data: input/output pairs centering on *apricot*

Asssume a +/- 2 word window

Skip-Gram Training with Negative Sampling

Training sentence:

... lemon, a tablespoon of **apricot** jam a pinch ...

c1

c2

t

c3

c4

positive examples +

t

c

apricot tablespoon

apricot of

apricot preserves

apricot or

- For each positive example, we'll create k negative examples.
- Any random word that isn't t

Skip-Gram Training with Negative Sampling

Training sentence:

... lemon, a tablespoon of **apricot** jam a pinch ...

c1

c2

t

c3

c4

positive examples +

t

c

apricot tablespoon

apricot of

apricot preserves

apricot or

negative examples - ^{k=2}

t

c

t

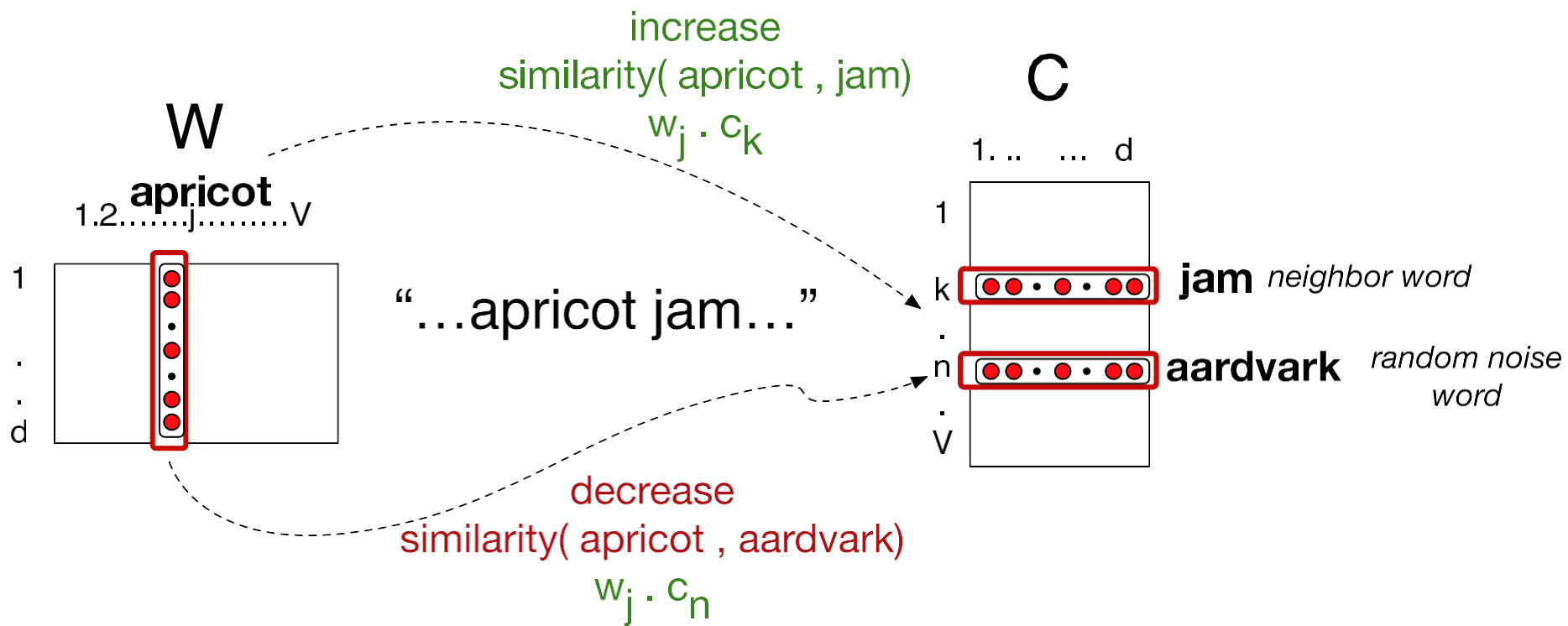
c

apricot aardvark apricot twelve

apricot puddle apricot hello

apricot where apricot dear

apricot coaxial apricot forever



How to compute $p(+ | t, c)$?

Intuition:

- Words are likely to appear near similar words
- Model similarity with dot-product!
- $\text{Similarity}(t, c) \propto t \cdot c$

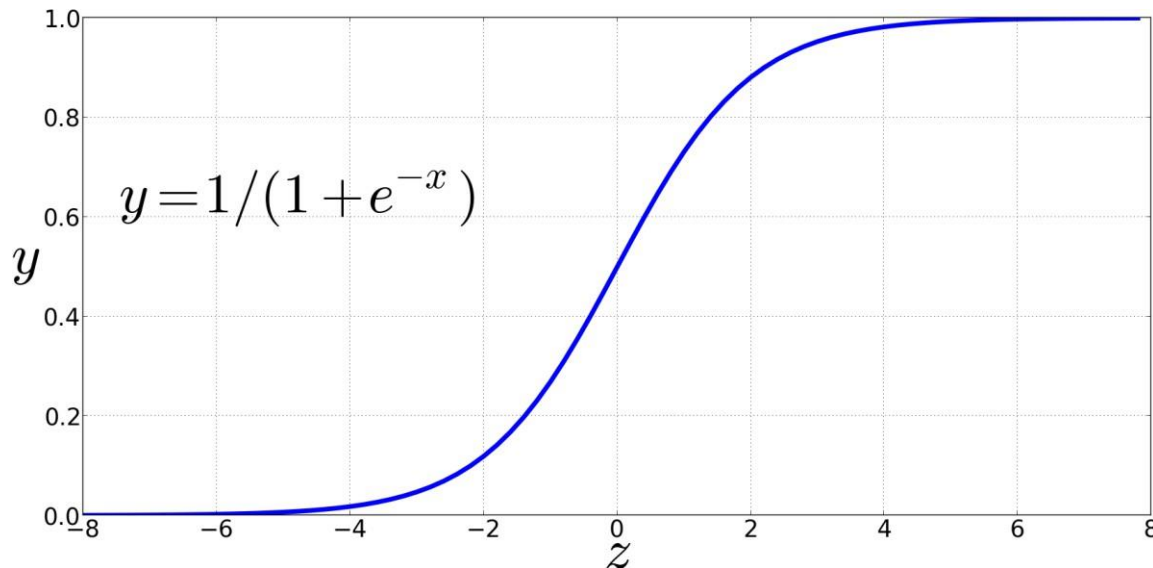
Problem:

- *Dot product is not a probability!*
 - *(Neither is cosine)*

Turning dot product into a probability

The sigmoid lies between 0 and 1:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Turning dot product into a probability

$$P(+|t, c) = \frac{1}{1 + e^{-t \cdot c}}$$

$$\begin{aligned} P(-|t, c) &= 1 - P(+|t, c) \\ &= \frac{e^{-t \cdot c}}{1 + e^{-t \cdot c}} \end{aligned}$$

For all the context words:

Assume all context words are independent

$$P(+|t, c_{1:k}) = \prod_{i=1}^{\kappa} \frac{1}{1 + e^{-t \cdot c_i}}$$

$$\log P(+|t, c_{1:k}) = \sum_{i=1}^k \log \frac{1}{1 + e^{-t \cdot c_i}}$$

Objective Criteria

We want to maximize...

$$\sum_{(t,c) \in +} \log P(+|t, c) + \sum_{(t,c) \in -} \log P(-|t, c)$$

Maximize the + label for the pairs from the positive training data, and the – label for the pairs sample from the negative data.

Loss Function: Binary Cross-Entropy / Log Loss

$$H_p(q) = -\frac{1}{N} \sum_{i=1}^N y_i \cdot \log(p(y_i)) + (1 - y_i) \cdot \log(1 - p(y_i))$$

Summary: How to learn word2vec (skip-gram) embeddings

Start with V random 300-dimensional vectors as initial embeddings

Use logistic regression, the second most basic classifier used in machine learning after naïve bayes

- Take a corpus and take pairs of words that co-occur as positive examples
- Take pairs of words that don't co-occur as negative examples
- Train the classifier to distinguish these by slowly adjusting all the embeddings to improve the classifier performance
- Throw away the classifier code and keep the embeddings.

Properties of embeddings

Similarity depends on window size C

(small window size, words that are syntactically similar and have same part of speech,

Large window size, words related but do not have same Part of speech)

$C = \pm 2$ The nearest words to *Hogwarts* (name of school):

- *Sunnydale* (name of fictional school)
- *Evernight* (name of fictional school)

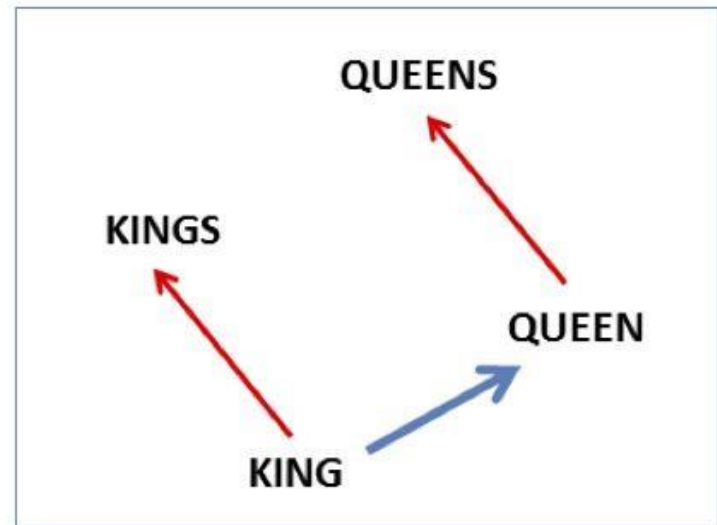
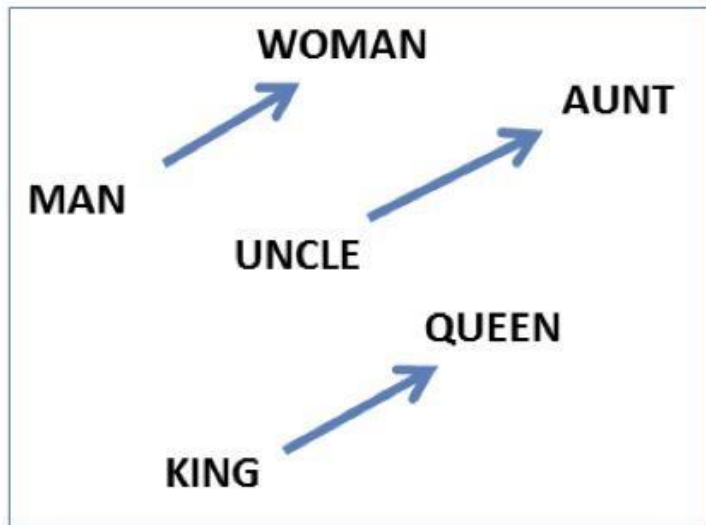
$C = \pm 5$ The nearest words to *Hogwarts*:

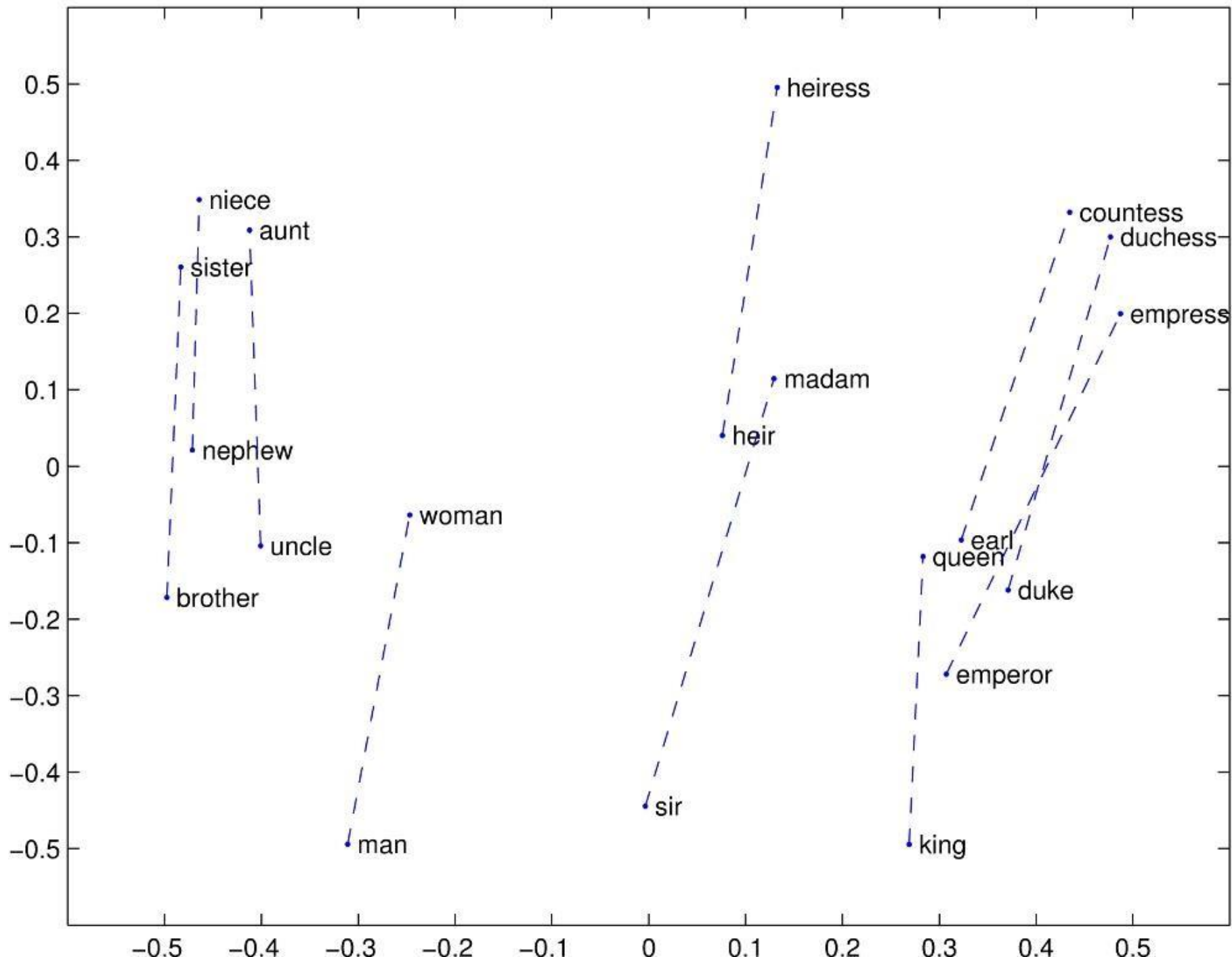
- *Dumbledore*
- *Malfoy*
- *halfblood*

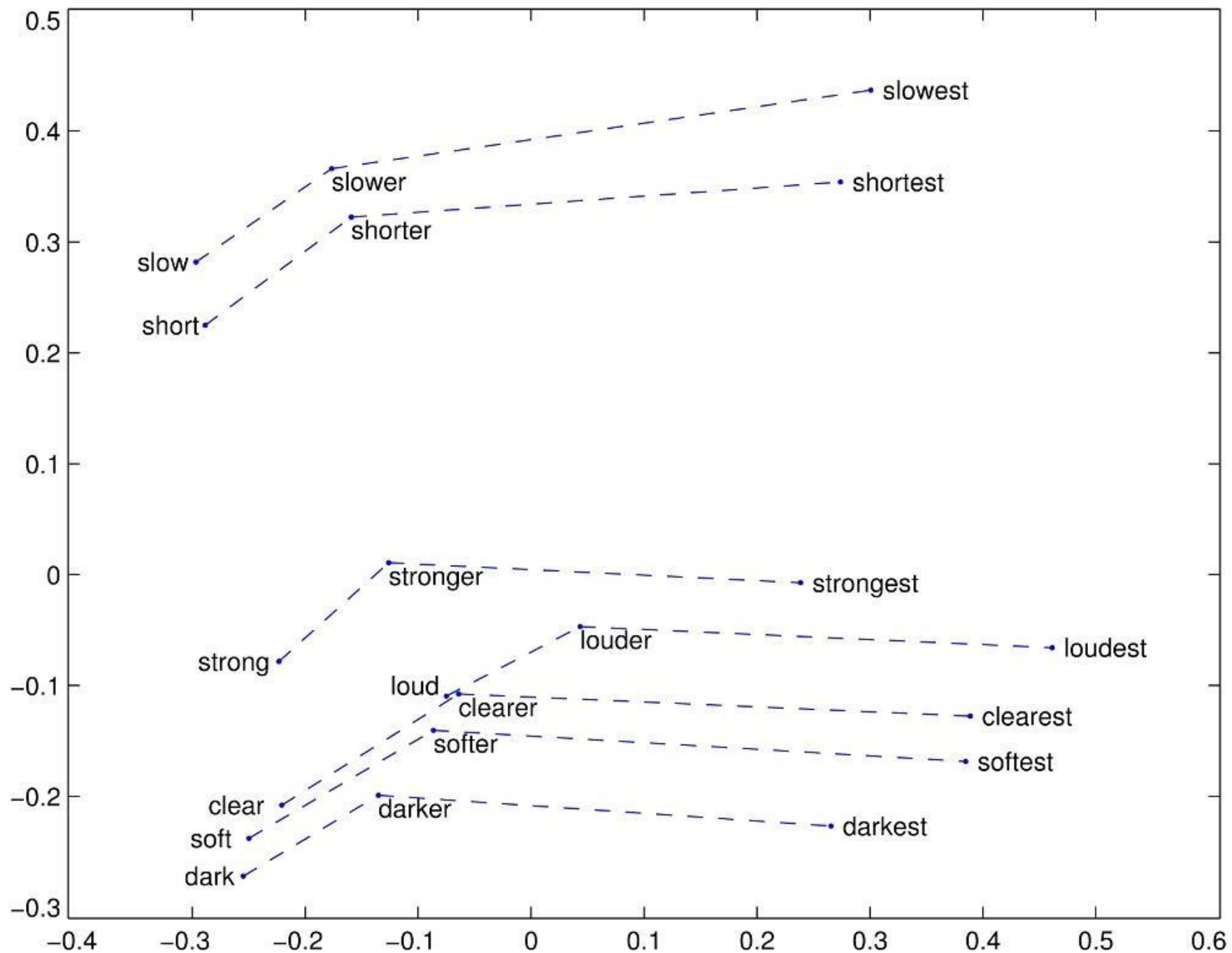
Analogy: Embeddings capture relational meaning!

$\text{vector}('king') - \text{vector}('man') + \text{vector}('woman') \approx \text{vector}('queen')$

$\text{vector}('Paris') - \text{vector}('France') + \text{vector}('Italy') \approx \text{vector}('Rome')$







Embeddings reflect cultural bias

Bolukbasi, Tolga, Kai-Wei Chang, James Y. Zou, Venkatesh Saligrama, and Adam T. Kalai. "Man is to computer programmer as woman is to homemaker? debiasing word embeddings." In *Advances in Neural Information Processing Systems*, pp. 4349-4357. 2016.

Ask “Paris : France :: Tokyo : x”

- x = Japan

Ask “father : doctor :: mother : x”

- x = nurse

Ask “man : computer programmer :: woman : x”

- x = homemaker

Conclusion

Concepts or word senses

- Have a complex many-to-many association with **words** (homonymy, multiple senses)
- Have relations with each other
 - Synonymy, Antonymy, Superordinate
- But are hard to define formally (necessary & sufficient conditions)

Embeddings = vector models of meaning

- More fine-grained than just a string or index
- Especially good at modeling similarity/analogy
 - Just download them and use cosines!!
- Can use sparse models (tf-idf) or dense models (word2vec, GLoVE)
- Useful in practice but know they encode cultural stereotypes

Dense embeddings you can download!

Word2vec (Mikolov et al.)

<https://code.google.com/archive/p/word2vec/>

Fasttext <http://www.fasttext.cc/>

Glove (Pennington, Socher, Manning)

<http://nlp.stanford.edu/projects/glove/>

Read from following links for WordToVec example

<https://towardsdatascience.com/an-implementation-guide-to-word2vec-using-numpy-and-google-sheets-13445eebd281>

https://nathanrooy.github.io/posts/2018-03-22/word2vec-from-scratch-with-python-and-numpy/?source=post_page-----13445eebd281

WordToVec Visualization

<https://ronxin.github.io/wevi/>

Readings

1. Dan Jurafsky and James Martin, Speech and Language Processing
Chapter 6: Vector Semantics
2. <https://towardsdatascience.com/an-implementation-guide-to-word2vec-using-numpy-and-google-sheets-13445eebd281>